Quick revision by Mishkat Mahabub(2AM)

Operator overloading is a programming feature that allows operators to be redefined and used with user-defined data types.

Member Function	Friend Function
1. Declared inside the class and inherently has access to its public, private and protected members.	Declared outside the class but has access to its private and protected members.
2.Called using an object or reference of the class.	Called like a regular function, not with an object of the class.

Rules of operator overloading:

- Only built-in operators can be overloaded. If some operators are not present in C++, we cannot overload them.
- The arity of the operators cannot be changed
- The precedence of the operators remains same.
- The overloaded operator cannot hold the default parameters except function call operator "()".
- We cannot overload operators for built-in data types. At least one user defined data types must be there.

why is it necessary to overload an operator?

Operator overloading is necessary to allow customized behavior of operators with user-defined types(like objects), enhancing the flexibility and readability of a programming language.

Limitations of using operator overloading:

1. Operator overloading, if not used carefully, can make code harder to understand and manage, causing confusion and ambiguity.
2. Not all operators, like member access operators (.) and (->), and the ternary operator (?:), can be overloaded.
3. Cannot Change Operator Precedence

4. Overloaded operators do not participate in polymorphism.

A friend function cannot be used in overloading assignment operator. explain why?

Direct Access Requirement:

- Assignment operator (=) needs direct access to class members to perform its task.
- Friend functions, being non-member functions, lack direct access to class members.

Modifying LHS object:

- the assignment operator is expected to modify the object on the left-hand side of the assignment operator (=), which is implicitly passed as *this.
- Friend function not having *this cannot modify LHS object directly.

Violation of Encapsulation:

- Overloading the assignment operator with a friend function would undermine encapsulation principles.

Therefore you must use member function to overload assignment operator in c++.

```
#include<iostream>
using namespace std;
class ass{
    int ft;
    int in;
public:
    ass():ft(0),in(0){}
    ass(int f, int i):ft(f),in(i){}
    ass operator =(const ass &ob){
        ft = ob.ft;
        in = ob.in;
    }
    void show(){
        cout<<ft<<" "<<in<<endl;
    }
};
int main(){
```

```
    ass ob1(6,7);
    ass ob2;
    ob2 = ob1;
    ob2.show();
}
```

Overload (&&) operator using friend function :

```
#include<bits/stdc++.h>
using namespace std;
class And {
   bool yes;
public:
   And(): yes(true){}
   And(bool f):yes(f){}
   friend bool operator &&(const And &ob1, const And &ob2);
};
bool operator &&(const And &ob1, const And &ob2){
      if(ob1.yes && ob2.yes){
         return true;
      }
      else return false;
   }
int main()
{
   And d1(false);
   And d2;
   if(d1 && d2){
      cout<<"YES"<<endl;
   }
   else cout<<"NO"<<endl;

}
```

Output : NO

```
int main()
```

```
{
   And d1(true);
   And d2;
   if(d1 && d2){
      cout<<"YES"<<endl;
   }
   else cout<<"NO"<<endl;

}
```
Output : YES.

Can the precedence of an operator be changed? Can the number of operand be altered?

:-

1. No, the precedence of an operator cannot be changed in C++. The precedence is fixed by the language and cannot be altered by overloading.
2. No, the number of operands for an operator cannot be changed. Overloading an operator must adhere to the same number of operands as defined by the operator's original functionality in C++.

For example (+) operator always takes 2 operands for its operation.

when an operator is overloaded, does it lose its original functionality?

No, when an operator is overloaded in C++, it does not lose its original functionality. Instead, the operator's functionality is extended to work with user-defined types. The original functionality remains intact and is used when the operator is applied to built-in types.

Example :

```
#include<bits/stdc++.h>
using namespace std;
class sum{
   int value;
public:
   sum():value(0){}
   sum(int i):value(i){}
   sum operator +(const sum &obj){
      int total;
```

```
    total = value + obj.value;
    return sum(total);
  }
  void show(){
    cout<< value <<endl;
  }
};
int main()
{
  sum d1(30),d2(9);
  sum d3;
  d3 = d1 + d2;// user defined types
  d3.show();
  int n1=25, n2=14;
  int n3 = n1 + n2;// built in types
  cout<<n3<<endl;
}
```

A derived class can access all the member of its base class. is this statement true?

No, the statement "a derived class can access all the members of its base class" is not entirely true. The derived class's access to the base class members depends on the access specifiers used in the base class.

- Public members of the base class are accessible to the derived class publicly.
- Protected members of the base class are accessible to the derived class privately.
- Private members of the base class are **not accessible** directly by the derived class. These members can only be accessed through public or protected member functions of the base class.

Example:

```
#include <iostream>
using namespace std;
class Base {
public:
int publicMember;
```

```
protected:
int protectedMember;
private:
int privateMember;
public:
Base():publicMember(1),protectedMember(2),privateMember(3){}

int getPrivateMember() const {
return privateMember; // Providing access to the private member through a public function
}
};
class Derived : public Base {
public:
void display() {
cout << "Public Member: "<<publicMember<<endl;      // Accessible
cout << "Protected Member: "<<protectedMember<<endl; // Accessible
//cout << "Private Member: "<<privateMember<<endl;  // Not accessible, would cause a compilation error
cout << "Private Member (via getter): "<<getPrivateMember()<<endl; // Accessible through public method
}
};

int main() {
Derived d;
d.display();
return 0;
}
```

Single inheritance:

```
#include <iostream>
using namespace std;
class Base {
public:
   void show() {
      cout<<"Base class show function"<<endl;
   }
};
class Derived : public Base {
public:
   void show() {
      cout<<"Derived class show function"<<endl;
   }
};
int main() {
   Base baseObj;
   Derived derivedObj;
   baseObj.show();  // Calls Base class show function
   derivedObj.show();  // Calls Derived class show function

   return 0;
}
```

in inheritance, explain why the protected category is needed?

:- The protected access specifier is needed in inheritance to balance data encapsulation with the ability of derived classes to use base class members. Here's why:

1. **Controlled Access for Derived Classes**: Protected members can be accessed by the base class and its derived classes but not by other parts of the program. This lets derived classes use important parts of the base class without making them public.
2. **Promotes Code Reusability and Extension** : Protected members let derived classes use and extend the base class's functionality, making it easier to reuse and enhance code.

Example:

```
#include <iostream>
using namespace std;
class Base {
protected:
   int protectedMember;
public:
   Base() : protectedMember(5) {}
   void display(){
      cout << "Base protectedMember: " << protectedMember << endl;
   }
};
class Derived : public Base {
public:
   void modifyProtectedMember(int value) {
      protectedMember = value; // Accessible due to protected access
   }
};

int main() {
   Derived d;
   d.display();             // Output: Base protectedMember: 5
   d.modifyProtectedMember(10);
   d.display();             // Output: Base protectedMember: 10
   return 0;
}
```

write a short program to implement multiple inheritance:

```
#include <iostream>
using namespace std;
class base1{
public:
   void showbase1(){
      cout<<"Base class 1"<<endl;
```

```
    }
};
class base2{
public:
    void showbase2(){
        cout<<"Base class 2"<<endl;
    }
};
class derived : public base1, public base2{
public:
    void show(){
        cout<<"Derived class"<<endl;
    }
};
int main()
{
    derived d;
    d.showbase1();
    d.showbase2();
    d.show();
}
```

What is Virtual base class? Explain with writing a program:

A virtual base class in C++, is a base class that ensures that only one instance of the base class is inherited by all derived classes in a multiple inheritance scenario, preventing duplication and ambiguity.

Virtual inheritance using virtual base class(diamond problem solve):

```
#include<bits/stdc++.h>
using namespace std;
class A{
public:
    void eat(){
        cout<<"Eat mangoe"<<endl;
    }
```

```
};
class B : virtual public A{
public:
   void donteat(){
      cout<<"dont Eat mangoe"<<endl;
   }
};
class C :  virtual public A{};
class D: public B, public C{};
int main(){
   D ob;
   ob.eat();
   ob.donteat();
}
```

Early Binding

Early binding (also known as **static binding** or **compile-time binding**) happens when the method or function to be called is determined at compile time. This means the decision about which function to invoke is made before the program runs.

Key points about early binding:

- The compiler knows exactly which function to call.
- It's typically faster because the decision is made at compile time.
- Examples include normal function calls and overloaded functions.

```
#include <iostream>

class Base {
public:
   void show() {
      std::cout << "Base class show function" << std::endl;
   }
};

class Derived : public Base {
public:
   void show() {
      std::cout << "Derived class show function" << std::endl;
```

```
    }
};

int main() {
    Base baseObj;
    Derived derivedObj;

    baseObj.show();  // Calls Base class show function
    derivedObj.show();  // Calls Derived class show function

    return 0;
}
```

How to invoke base class parameterized constructor inside derived class's parameterized constructor? Example:

```
#include <iostream>
using namespace std;
class Base {
   int value;
public:
   Base():value(0){ cout<<"Base"<<endl;}
   Base(int v): value(v){cout<<"Base Value "<<value<<endl;}
};
class Derived : public Base {
public:
   Derived():Base(){cout<<"Derived"<<endl;};
   Derived(int v): Base(v){cout<<"Derived value "<<endl;}
};
int main() {
   Derived d1, d2(10);
   return 0;
}
```

Output :

```
Base
Derived
Base Value 10
Derived value 10
```

Write the output for the following code:

```
class A
{
   public:
   void cheers()
   {
      cout<<"Class A: Hip-hip-hooray";
   }
};
class B
{
   public:
   void cheers()
   {
      cout<<"Class B: Hip-hip-hooray";
   }
};
class C:public A, public B
{
};
int main()
{
   C obc;
   obc.cheers();
}
```

Is there any error in this code? If yes, then correct the code. Display the output.

The provided code has an issue due to ambiguity caused by multiple inheritance. To resolve this ambiguity, we can explicitly specify which **cheers** method we want to call using the scope resolution operator. Here is the corrected version of the code:

```
#include <iostream>
using namespace std;

class A {
public:
   void cheers() {
      cout << "Class A: Hip-hip-hooray" << endl;
   }
```

```
};

class B {
public:
   void cheers() {
      cout << "Class B: Hip-hip-hooray" << endl;
   }
};

class C : public A, public B {
};
int main() {
   C obc;
   obc.A::cheers();  // Explicitly call cheers() from class A
   obc.B::cheers();  // Explicitly call cheers() from class B
   return 0;
}
```

Difference between operator functions and normal functions:

1. **Overloading**: Operator functions allow you to overload operators like +, -, *, etc., to work with custom types. Normal functions don't have this capability.
2. **Syntax**: Operator functions use symbols (e.g., +, -, *) as their names, making code more readable for operations like addition, subtraction, and multiplication. Normal functions require explicit names.
3. **Implicit call**: Operator functions can be called implicitly by using operators with operands of compatible types, while normal functions must be explicitly called with their names and arguments.

Write a program using the following statement properly with a C++ program:
Ob2 = 20 + Ob1

```
#include<bits/stdc++.h>
using namespace std;
class Sum{
   int value;
public:
   Sum() : value(0){}
```

```
    Sum(int v) : value(v){}
    Sum operator +(const Sum &obj){
        return Sum(value + obj.value);
    }
    void show(){
        cout<<value<<endl;
    }
    friend Sum operator+(int a, const Sum &obj);
};
Sum operator +(int a,const Sum &obj){
        return Sum(a + obj.value);
    }
int main()
{
    Sum obj1(10),obj2;
    obj2 = 29 + obj1;
    obj2.show();
}
```

write a class named fruit with a data memeber to calculate the number of fruits basket. Create two other class named apple and mango tocalculate the number of apples and mangoes in the busket. print the number of fruits in each basket and total number of the fruits in the busket. using inheritance

```
#include<bits/stdc++.h>
using namespace std;
class fruit{
protected:
    int num;
public:
    fruit():num(0){}
    int total(){
        return num;
    }
};
class apple : public fruit{
public:
    apple(int cnt){
```

```
        num = cnt;
    }

};
class mangoe: public fruit{
public:
    mangoe(int cnt){
        num = cnt;
    }

};
int main(){
    apple a(10);
    mangoe m(29);

    cout<<"Number of apples "<<a.total()<<endl;
    cout<<"Number of mangoes "<<m.total()<<endl;
    cout<<"total number of fruits "<<a.total()+m.total()<<endl;
}
```

Late Binding

Late binding (also known as **dynamic binding** or **run-time binding**) happens when the method or function to be called is determined at runtime. This means the decision is made while the program is running, usually through mechanisms like virtual functions in object-oriented programming.

Key points about late binding:

- The exact function to call is determined at runtime.
- It's more flexible and allows for polymorphism.
- It might be slightly slower due to the overhead of determining the function to call at runtime.

```
#include <iostream>

class Base {
public:
    virtual void show() {
        std::cout << "Base class show function" << std::endl;
```

```
    }
    virtual ~Base() {}  // Virtual destructor for proper cleanup
};

class Derived : public Base {
public:
    void show() override {
        std::cout << "Derived class show function" << std::endl;
    }
};

int main() {
    Base* basePtr;
    Derived derivedObj;

    basePtr = &derivedObj;
    basePtr->show();  // Calls Derived class show function due to late binding
(polymorphism)

    return 0;
}
```

Adding two number (int & float) using template :

```
#include <iostream>
using namespace std;

template <class T>
T sum(T a, T b) {
    return a + b;
}

int main() {
    cout << sum<double>(10, 20) << endl;
    return 0;
}
```

Virtual Function:

- A virtual function is a function that can be overridden in a derived class. This allows the derived class to provide its own implementation of the function.

Why do we need pure virtual function:

- Virtual functions are needed to achieve runtime polymorphism, allowing different objects of derived classes to be treated uniformly through a common base class interface

Pure virtual function:

A pure virtual function is a function declared in a base class with the syntax = 0, which must be overridden in any derived class, making the base class abstract and preventing its instantiation.

Why do we need to use pure virtual function?

1. **Preventing instantiation of abstract classes:** A pure virtual function has no implementation, making the class abstract. This ensures objects of the abstract class can't be created, which prevents meaningless instances.
2. **Forcing derived classes to implement certain functions**: If a base class has a pure virtual function, any derived class must implement it. This ensures all derived classes have specific functionality.
3. **Improving type safety**: Pure virtual functions ensure the compiler only allows them to be called on appropriate object types, preventing errors from using the wrong type.

What is polymorphism?

In object-oriented programming (OOP), polymorphism refers to the ability of objects of different classes to be treated as objects of a common superclass.

Difference between Early binding and Late binding:

feature	Early binding	Late binding
Type of polymorphism	static	Dynamic
Time of type determination	Compile time	Run time
speed	Faster	Slower
flexibility	Less flexible	More flexible
Debugging	Easier	Harder

Example of pure virtual function :

```
#include<bits/stdc++.h>
using namespace std;
class A{
   public:
   virtual void show()=0;
};
class B : public A{
public:
   void show()override{
      cout<<"i am in B"<<endl;
   }
};
class C : public A{
   public:
   void show()override{
      cout<<"I am in C"<<endl;
   }
};
int main()
{
   B b;
   C c;
   b.show();
   c.show();
}
```

Static polymorphism:

```
#include <iostream>
using namespace std;
class Print {
public:
   void show(int i) {
      cout << "Integer: "<<i<<endl;
   }

   void show(double d) {
```

```
        cout<<"Double: "<<d<<endl;
    }
};
int main() {
    Print p;
    p.show(39);    // Calls show(int)
    p.show(3.9);  // Calls show(double)
    return 0;
}
```

Dynamic polymorphism:

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual void display() {
        cout << "Display from Base" << endl;
    }
};

class Derived : public Base {
public:
    void display() override {
        cout << "Display from Derived" << endl;
    }
};


int main() {

    Base b;
    b.display(); // display from base
    Derived d;
    d.display();// display from derived
    return 0;
}
```

Manipulator:

```
#include <iostream>
using namespace std;

ostream &display(ostream &output){
   output.width(10);
   output.precision(2);
   output.setf(ios::fixed| ios::right);
   output.fill('*');
   return output;
}
int main(){
   cout<<display<<23300<<endl;
   return 0;
}
```

Define exception handling:

Exception handling in C++ allows you to manage errors that occur during program execution.

Benefits of exception handling:

1. **Dealing with Mistakes**: Exception handling helps deal with mistakes that happen when the program runs.
2. **Stronger Programs**: It makes programs stronger by handling mistakes without crashing.
3. **Clear Code**: It keeps the code clean by separating mistake from regular code.
4. **Organized Fixes**: Helps fix mistakes in a systematic way, making code easier to manage.
5. **Easy to Find Errors**: Makes it easier to find and fix mistakes while working on the program.
6. **Good with Resources**: Helps manage computer resources (like memory) well, avoiding wastage.

STREAM :

A stream is a sequence of bytes representing data flowing between a program and an external device, such as the console, files, or network connections.

```
#include<bits/stdc++.h>
```

```
using namespace std;
int main()
{
    ofstream outfile("whoareyou.txt");
    string name, sem, coursetitle, coursecode;
//    getline(cin,name);
//    getline(cin,sem);
//    getline(cin,coursecode);
//    getline(cin,coursetitle);
    name = "xxxxxxx \n";
    sem = "Spring-2022 \n";
    coursecode = "CSE-1221 \n";
    coursetitle = "Computer Programming II \n";

    outfile<<"Name : "<<name<<"Semester : "<<sem<<"Course Code :
"<<coursecode<<"Course Title : "<<coursetitle;
    outfile.close();

    ifstream infile("whoareyou..txt");
    infile>>name>>sem>>coursecode>>coursetitle;
    cout<<name;
    cout<<sem;
    cout<<coursecode;
    cout<<coursetitle;
    infile.close();
}
```